

Finalized 블록 기준 처리와 파이프라인 E2E 검증

NFTIssuedProcessor · DeferredProcessingError · M2 전체 파이프라인 E2E
강의 55min + 실습 30min

NFTIssuedProcessor

Finalized 체크

M2 E2E 완주

COINCRAFT
ACADEMY

FinalizedBlockProvider — VASP 연동에서 필요한가

FINALIZED BLOCK CHECK IN VASP-DELEGATED ARCHITECTURE

Phase별 역할 분담

Phase	TX 처리 주체	FinalizedBlockProvider
Phase 1	외부 VASP	미주입 — 스킵
Phase 2	외부 VASP + 직접 구독	선택적 활성화
Phase 3	Kyobo Custody (HSM/MPC)	필수 주입

Phase 1 트레이드오프 — VASP SLA가 finalization을 보장하면 이중 체크 불필요. 보장하지 않으면 Reorg 위험을 VASP에 위임.

optional 설계 이유 (?)

```
// Phase 1 - FinalizedBlockProvider 미주입
// VASP 신뢰, finalization 체크 스킵
const processor = new NFTIssuedProcessor(
    idempotency, ledger
);

// Phase 3 - FinalizedBlockProvider 주입
// 직접 온체인 조회, 미확정 블록 → Deferred
const processor = new NFTIssuedProcessor(
    idempotency, ledger,
    evmAdapter // getFinalizedBlockNumber()
);
```

구현체 교체 비용 0 — 상위 레이어 수정 없이 생성자 인자 하나만 추가. Phase 1 → 3 전환이 이 줄 하나.

M2 실습 기본값 — FinalizedBlockProvider 미주입. Finalized 체크 없이 전체 파이프라인 E2E 동작 확인이 목표. Phase 3 전환 시 EVMAdapter 주입만으로 활성화.

왜 Finalized 블록 기준인가 — 블록 확정 단계

WHY FINALIZED — BLOCKCHAIN REORG AND FINALITY

블록 확정 3단계

단계	상태	안전성
PENDING	아직 마이닝 안 됨	처리 금지
CONFIRMED	마이닝됨, but Reorg 가능	위험
FINALIZED	PoS Checkpoint 통과, 절대 불변	처리 가능

Ethereum PoS Finality — 블록 생성 후 2 epochs(약 12분) 뒤 Finalized. 약 64블록 이상.

Reorg(체인 재편)가 일어나면

블록 #100에 NFT 발행 TX 포함

- 원장 업데이트 완료 (CONFIRMED에서 처리)
- 체인 재편: 블록 #100이 고아 블록(Orphan)
- TX가 없던 일이 됨
- 원장에는 NFT 발행 기록 남음 (불일치!)

Finalized 이전에 처리하면:

- = 존재하지 않는 NFT 발행 기록
- = 회계 불일치 = 감사 실패

금융 시스템에서 **Finalized** 검증은 필수 — 미확정 블록 처리 = 잠재적 손실.

Phase 1: IBlockchainAdapter.getReceipt()로 read-only 조회 방식.

NFTIssuedProcessor — 구조와 의존성

NFT_ISSUED EVENT PROCESSOR — THREE DEPENDENCIES

클래스 구조 (processors/NFTIssuedProcessor.ts)

```
export class NFTIssuedProcessor
  implements EventProcessor {

  readonly eventTypes = ['NFT_ISSUED'];

  constructor(
    private readonly idempotency:
      IdempotencyGuard,          // 필수
    private readonly ledger:
      LedgerService,            // 필수
    private readonly finalizedBlockProvider?:
      FinalizedBlockProvider,    // 선택
  ) {}
}
```

3개 의존성 상세

의존성	역할	필수?
IdempotencyGuard	중복 처리 차단 (run + key)	필수
LedgerService	원장 업데이트 (creditNFT)	필수
FinalizedBlockProvider	블록 확정 여부 조회	선택 (?)

선택적 의존성 설계 이유 — Finalized 검증 없이도 동작 가능 (M2 실습 기본값). 점진적 기능 추가에 유리. 테스트에서 mock 교체 용이.

DIP 원칙 — 의존성이 모두 인터페이스 → M4에서 LedgerService를 PostgreSQL 구현체로 교체해도 이 클래스 코드 변경 0.

의존성 주입과 DIP — 인터페이스가 미래 교체 비용을 0으로 만든다

DEPENDENCY INJECTION & DIP — ZERO-COST FUTURE REPLACEMENT

인터페이스 2개

```
export interface LedgerService {
  creditNFT(owner: string, tokenId: string,
    amount?: number): Promise<void>;
  getNFTBalance(owner: string, tokenId: string): Promise<number>;
}

export interface FinalizedBlockProvider {
  getFinalizedBlockNumber(): Promise<number>;
}
```

FinalizedBlockProvider — 블록체인 finalized 블록 번호 반환. 이 번호 이하 블록은 "절대 뒤집히지 않음" 보장.

M2 → M4 교체 비용 0

```
// 현재 (M2 실습): 메모리 구현
const ledger: LedgerService =
  new InMemoryLedgerService();

// M4 (PostgreSQL): 구현체만 교체
const ledger: LedgerService =
  new PostgresLedgerService(pool);

// NFTIssuedProcessor 코드는 동일 - 인터페이스만 만족
```

DIP (Dependency Inversion Principle) — 고수준 모듈(processor)이 저수준 모듈(DB)에 직접 의존하지 않고 추상(인터페이스)에 의존.

콜백 패턴 멍등성 — `idempotency.run(key, callback)`: 키 체크 + 처리를 캡슐화. 사용자는 "할 일"만 작성. 내부 구현 변경에 독립적.

process() — 3단계 처리 라인별 해부

THREE-STEP PROCESSING: FINALITY → IDEMPOTENCY → LEDGER

```
async process(message: StreamMessage): Promise<void> {
  const payload = JSON.parse(message.fields['payload'] ?? '{}');
  const requestId = message.fields['requestId'] ?? message.id;
  // requestId fallback: 명시적 ID → Redis messageId

  // Step 1: Finalized 블록 검증
  if (this.finalizedBlockProvider && payload.blockNumber != null) {
    const finalized = await this.finalizedBlockProvider
      .getFinalizedBlockNumber();
    if (payload.blockNumber > finalized) {
      throw new DeferredProcessingError(
        `block ${payload.blockNumber} not yet finalized`
      );
    }
    // retryCount 증가 없음 - "실패"가 아닌 "자연"
  }

  // Step 2+3: 멍등성 확인 → 원장 업데이트
  await this.idempotency.run(`NFTIssued:${requestId}`, async () => {
    await this.ledger.creditNFT(payload.to, payload.tokenId);
  });
}
```

3단계 처리 순서표

단계	동작	XACK 여부
Finalized 미확정	DeferredProcessingError throw	없음 → PEL 잔류 (지연)
중복 requestId	idempotency.run() 스킵	Worker가 처리
정상 처리	creditNFT() 실행	Worker가 XACK

중요 — XACK는 NFTIssuedProcessor가 호출하지 않음.
ConsumerGroupWorker._handleWithRetry()에서 처리. 관심사 분리.

requestId fallback — 외부 시스템이 requestId 안 보낼 수 있음. Redis messageId는 고유값 보장 → 차선책으로 충분.

blockNumber vs Finalized — 처리 판단 기준

BLOCK FINALITY CHECK — PROCESS OR DEFER?

블록체인 — 시간 흐름 →



✓ `blockNumber ≤ finalized` → 처리 가능



▲
`finalized`
= 995



|| `blockNumber > finalized` → `DeferredProcessingError`

✓ `payload.blockNumber ≤ finalized`

```
payload.blockNumber = 993
finalized           = 995
```

```
// 993 ≤ 995 - 이미 확정된 블록, Reorg 불가
await ledger.creditNFT(payload.to, payload.tokenId);
```

원장 업데이트 완료 → Worker가 XACK → 메시지 완료

|| `payload.blockNumber > finalized`

```
payload.blockNumber = 998
finalized           = 995
```

```
// 998 > 995 - Reorg으로 뒤집힐 수 있음
throw new DeferredProcessingError(`block 998 not finalized`);
```

XACK 없음 → PEL 잔류 → `finalized` 이후 자동 재처리

DeferredProcessingError — "실패"와 "지연"의 구분

DEFERRED vs ERROR — CRITICAL DESIGN DISTINCTION

throw 방식별 Worker 반응

상황	throw 방식	retryCount	결과
DB 연결 실패	<code>new Error(...)</code>	+1	3회 후 DLQ
외부 API 타임아웃	<code>new Error(...)</code>	+1	3회 후 DLQ
블록 미확정	<code>new DeferredProcessingError</code>	유지	PEL 잔류 → 자동 처리

왜 **return**이 아닌 **throw**? — **return**하면 Worker가 성공으로 간주 → XACK 호출 → 메시지 영구 소실. **throw** `DeferredProcessingError` → Worker가 특수 분기 진입 → PEL 잔류.

Worker 측 처리 (ConsumerGroupWorker.ts)

```
try {
  await processor.process(msg);
  await xack(...); // 성공
} catch (err) {
  if (err instanceof DeferredProcessingError) {
    // 카운트 안 올림, XACK 안 함
    // 시간 지나면 _reclaimPending()이 재수신
    return;
  }
  // 일반 에러 → retryCount 증가
  msg.fields['_retryCount'] = String(retryCount + 1);
}
```

핵심 통찰 — DB 실패 = 시스템 문제 → 빨리 알아채야 함 → DLQ. 블록 미확정 = 시간 지나면 해결 → 기다리면 됨. 같은 throw지만 의미가 다르다.

이 세션에서 새로 등장한 TypeScript 문법

NEW TYPESCRIPT PATTERNS IN S12

implements EventProcessor

클래스가 인터페이스를 만족함을 명시. 필드/메서드 모두 구현해야 컴파일 통과. 단순 형태 맞추기가 아닌 명시적 약속.

readonly eventTypes

한 번 정해지면 변경 불가. 외부에서 `processor.eventTypes = [...]` 시도 → 컴파일 에러. 사실상 상수.

private readonly 생성자

별도 필드 선언 없이 생성자 매개변수에서 바로 `this.x = x` 처리. TS 보일러플레이트 절약.

선택적 의존성 ?:

`finalizedBlockProvider?: FinalizedBlockProvider`
없으면 `undefined`. `Finalized` 검증 없이도 동작. 점진적 기능 활성화.

!= null 관용

`payload.blockNumber != null`
느슨한 비교로 `null`과 `undefined` 둘 다 검사. `!= null && != undefined`의 단축형. 의도적 예외.

throw new DeferredProcessingError

커스텀 에러 클래스. `Worker`가 `instanceof`로 잡아 특별 처리. 일반 `Error`와 의미 구분. "실패"와 "자연" 분리.

idempotency.run(key, callback)

함수를 인자로 넘기는 콜백 패턴. `run()`이 키 체크 후 신규일 때만 콜백 실행. 멍등성 로직 캡슐화.

JSON.parse(str ?? '{}')

`payload` 없으면 빈 객체 `JSON`. 파싱 결과는 `{}` → `undefined` 접근 안전. 잘못된 `JSON`이면 `throw` → `DLQ`.

requestId ?? message.id

우선순위 fallback. 비즈니스 ID → 큐 자동 생성 ID. `Redis message.id`는 고유 보장. 멍등성 키로 충분.

멍등성 키 네임스페이스

`NFTIssued:${requestId}` — 이벤트 타입 접두사로 충돌 방지. `NFTBurned`와 `NFTIssued`는 같은 `requestId`도 별도 키.

M2 파이프라인 전체 — 한눈에 보기

M2 FULL PIPELINE — END-TO-END REVIEW



M2 핵심 — 각 레이어는 인터페이스만 바라봄. WebhookServer는 Handler를 모르고, Worker는 Processor 내부를 모르고, Processor는 LedgerService 구현체를 모름.

M2 모듈 완료 기준 체크리스트 + 강의 강조 포인트

M2 COMPLETION CRITERIA & KEY TAKEAWAYS

M2 완료 기준 (6가지)

- **At-least-once + 멍등성** — 동일 이벤트 2회 → 원장 1회만 반영
- **장애 복구** — Consumer 강제 종료 → 재시작 후 미ACK 자동 재수신
- **DLQ** — 3회 실패 → DLQ 이동 + 알림
- **Finalized 체크** — 미확정 이벤트 → 처리 보류 (XACK 안 함)
- **E2E** — Webhook → Stream → Consumer → 원장 1건 완주
- **수평 확장** — Consumer 2개 동시 → 중복 처리 없음

S12 강의 강조 포인트

- **implements로 명시적 계약** — 시그니처 어긋나면 컴파일 에러
- **readonly 클래스 필드** — 인스턴스 생성 후 변경 불가
- **DeferredProcessingError 패턴** — "재시도 실패"와 "지연"을 구분
- **블록체인 reorg 인식** — 금융 시스템에서 Finalized 검증 필수
- **멍등성 키 네임스페이스** — EventType:requestId 형식으로 충돌 방지
- **인터페이스 의존 = 미래 교체 비용 0** — M4에서 LedgerService 교체해도 그대로
- **3단계의 미묘한 차이** — Finalized=사전 조건, 멍등성=중복 방지, 원장=실제 효과

M2 이벤트 파이프라인의 신뢰성 3원칙

THREE RELIABILITY PRINCIPLES OF M2 EVENT PIPELINE

원칙 1

At-least-once + 멍등성

"최소 1회 처리 보장
+ 중복 발행 차단"

DB unique constraint
+ ON CONFLICT DO NOTHING

→ 중복 수신해도
원장은 1회만 반영

S9에서 설계 → S10에서 구현

원칙 2

PEL + XAUTOCLAIM

"Consumer 장애 시
미처리 메시지 자동 복구"

XACK를 마지막 단계로
→ PEL에 잔류
→ 30초 후 재수신

멍등성이 중복 차단
→ 재처리해도 안전

S7 CLI 실습 → S10 코드 구현

원칙 3

DLQ + 알림

"3회 실패 메시지 격리
+ 운영자 개입 유도"

무한 재시도 대신 격리
→ 원인 파악
→ 수동 `requeueMessage`

큐 막힘 없음
운영자가 즉시 인지

S11 DLQ 설계 → S10 연결

M3 예고 — M2: "이벤트가 도착했을 때 어떻게 안전하게 처리하는가" → M3: "TX를 보냈을 때 블록체인이 어떤 경로로 실패하는가, 각각 어떻게 복구하는가"
(TxStateMachineService 7-상태 머신)

M2 → M3 연결 고리 · S12 실습 안내

FROM M2 TO M3 — HOW EVERYTHING CONNECTS

M2에서 배운 것이 M3에서 재사용되는 곳

M2에서 배운 것	M3에서 다시 등장
ConsumerGroupWorker At-least-once	S22: pollStaleRequests 결과 처리
IdempotencyGuard requestId 중복 차단	S16: VASP 재전송 방어
DLQHandler 3회 실패 격리	S17: NonRetryableError → 즉시 DLQ
Finalized 블록 기준 처리	S13: MINED → FINALIZED → CONFIRMED 전이

S12 실습 — S12_e2e.ts

실행 — event-engine 폴더에서

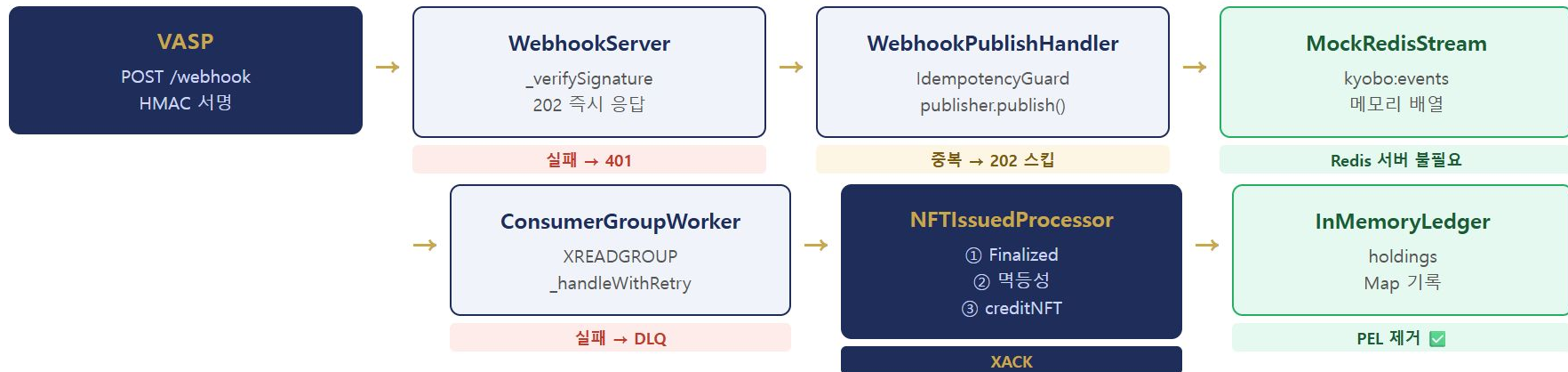
```
npx ts-node src/exercises/S12_e2e.ts
```

- **TODO 1** — WebhookPublishHandler + server.on 등록
- **TODO 2** — NFTIssuedProcessor 생성
- **TODO 3** — ConsumerGroupWorker 생성 (batchSize=10, blockMs=30)
- **TODO 4** — 동일 body 2번 전송 → 멍등성 확인
- **TODO 5** — invalid-signature → 401 확인

M2 완주 기준 — 5개 검증 모두 ☒ = M2 이벤트 파이프라인 신뢰성 3원칙 구현 완료.

S12 실습 — MockRedisStream으로 E2E 파이프라인 완주

END-TO-END FLOW — NO REAL REDIS NEEDED



TODO 1 · 2

- WebhookPublishHandler 생성
- server.on('NFT_ISSUED', handler)
- NFTIssuedProcessor 생성

TODO 3 · 4

- ConsumerGroupWorker 생성
- streamKey='kyobo:events'
- 동일 requestId 재전송 (먹등성 검증)

TODO 5

- 잘못된 서명으로 전송
- 기대값: 401
- HMAC 검증 실패 경로 확인

5가지 E2E 검증 시나리오

FIVE VERIFICATION SCENARIOS — S12_e2e.ts

#	검증 항목	확인 방법	기대값	TODO
1	정상 Webhook 수신	HTTP 응답 코드 확인	202	WebhookPublishHandler 생성 + server.on 등록
2	Stream 적재 확인	mockRedis.messageCount('kyobo:events')	1건	—
3	원장 업데이트	ledger.getNFTBalance('0xABCD', '42')	1	NFTIssuedProcessor 생성
4	역등성 검증 (동일 requestId 재전송)	Stream 수 + 원장 잔고 재확인	여전히 1 / 1	같은 body + sig로 2번째 전송
5	HMAC 검증 실패	잘못된 서명으로 전송	401	invalid-signature로 전송

TODO 목록 — TODO 1: WebhookPublishHandler + server.on / TODO 2: NFTIssuedProcessor / TODO 3: ConsumerGroupWorker (streamKey='kyobo:events', batchSize=10, blockMs=30) / TODO 4: 동일 requestId 재전송 / TODO 5: 잘못된 서명 전송